

Établissement		Classe / Groupe	
Nom et prénom		Date	

SUJET PROPOSÉ - TP WEB DYNAMIQUE

Dynamisation du template e-commerce FoodMart

Durée indicative 2 h 30	Barème 20 points	Technologies PHP 8, MySQL, PDO
----------------------------	---------------------	-----------------------------------

1. Contexte

Le dossier **food2** contient un template e-commerce FoodMart entièrement statique. Les catégories visibles dans la section « Category » sont écrites directement dans le fichier HTML. Toute modification impose donc de modifier manuellement la page.

Votre travail consiste à transformer cette partie du site en une version dynamique : les données devront être lues depuis une base MySQL et affichées par PHP, tout en conservant exactement le design fourni.

2. Objectifs pédagogiques

- Distinguer un affichage statique d'un affichage dynamique.
- Créer et alimenter une base de données MySQL.
- Se connecter à MySQL avec PDO.
- Exécuter une requête SELECT et récupérer plusieurs lignes.
- Afficher les résultats dans une page HTML avec une boucle PHP.
- Organiser le code en séparant connexion, fonctions et affichage.

3. Travail demandé

3.1 - Préparation du projet

- Décompresser le dossier food2 et le renommer food.
- Renommer index.html en index.php.
- Conserver les dossiers css, js, images et assets sans modifier le rendu graphique.
- Créer le dossier inc destiné aux fichiers PHP réutilisables.

3.2 - Création de la base de données

Créer une base nommée TpMagasin, puis une table categorie respectant la structure suivante :

```
CREATE TABLE categorie (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100) NOT NULL,
    image VARCHAR(255) NOT NULL
);
```

- Insérer au minimum les douze catégories présentes dans le template original.
- Dans le champ image, enregistrer le chemin relatif de l'image, par exemple images/icon-vegetables-broccoli.png.

3.3 - Connexion PDO

- Créer le fichier inc/connexion.php.
- Écrire une fonction connection() qui retourne un objet PDO valide.
- Activer PDO::ATTR_ERRMODE avec PDO::ERRMODE_EXCEPTION.
- Ne laisser aucun message de débogage tel que « coucou » ou « réussi » dans la version finale.

3.4 - Lecture des catégories

- Créer le fichier inc/fonction.php.
- Écrire une fonction getAllCategories() qui exécute SELECT id, nom, image FROM categorie ORDER BY id.
- Retourner toutes les lignes sous forme de tableau associatif avec PDO::FETCH_ASSOC.
- Ne pas mélanger PDO et MySQLi : fetch_assoc(), bind_param() et get_result() sont interdits dans cette solution PDO.

3. Travail demandé - suite

3.5 - Affichage dynamique dans index.php

- Inclure le fichier inc/fonction.php dans index.php.
- Appeler getAllCategories() avant la section « Category ».
- Supprimer les catégories écrites en dur dans le HTML.
- Utiliser une boucle foreach pour générer chaque élément du carrousel.
- Afficher le nom et l'image provenant de la base de données.
- Utiliser htmlspecialchars() pour protéger les valeurs affichées.

Structure attendue pour chaque catégorie :

```
<?php foreach ($categories as $categorie): ?>
  <a class="nav-link category-item swiper-slide">
    "
    <h3><?= htmlspecialchars($categorie['nom']) ?></h3>
  </a>
<?php endforeach; ?>
```

3.6 - Vérifications obligatoires

Test à effectuer	Résultat attendu
Ajouter une catégorie uniquement dans MySQL	La nouvelle catégorie apparaît après actualisation, sans modifier index.php.
Modifier le nom d'une catégorie dans MySQL	Le nouveau nom est visible sur la page.
Supprimer une catégorie dans MySQL	La catégorie disparaît du carrousel.
Arrêter MySQL ou utiliser un mauvais mot de passe	Une erreur est gérée proprement ; aucune page blanche incompréhensible.
Inspecter le HTML/PHP	Aucune liste complète de catégories n'est encore codée statiquement.

4. Contraintes techniques

- Technologies imposées : PHP, MySQL ou MariaDB et PDO.
- Le rendu visuel du template FoodMart doit être conservé.
- Les catégories ne doivent plus être écrites manuellement dans index.php.
- Le code SQL doit être fourni dans un fichier base.sql.
- Les erreurs PHP doivent être corrigées : utiliser \$conn->prepare(\$sql), puis execute() et fetchAll().
- Les chemins d'images stockés en base doivent être relatifs au projet.
- Le projet doit fonctionner après importation de base.sql et adaptation éventuelle des identifiants MySQL.

4.1 - Structure minimale attendue

```
food/
|-- index.php
|-- base.sql
|-- inc/
|   |-- connexion.php
|   |-- fonction.php
|-- css/
|-- js/
|-- images/
`-- assets/
```

- Les noms peuvent varier légèrement, mais la séparation des responsabilités doit rester claire.

5. Questions de compréhension

1. Pourquoi food2 est-il considéré comme un projet statique ?
2. Quel est le rôle de PHP dans la nouvelle version ?
3. Quel est le rôle de PDO ?
4. Quelle différence existe entre prepare(), execute() et fetchAll() ?
5. Pourquoi ne faut-il pas mélanger PDO avec les fonctions MySQLi ?
6. Comment vérifier que la liste affichée provient réellement de la base ?
7. Qu'est-ce qu'AJAX et dans quel cas pourrait-il être ajouté à ce projet ?

6. Barème indicatif

Élément évalué	Critères principaux	Points
Base de données	Table correcte, données et chemins d'images	4
Connexion PDO	Connexion fonctionnelle et gestion des erreurs	3
Fonction de lecture	SELECT, prepare, execute, fetchAll	4
Intégration dynamique	Boucle PHP, design conservé, aucun contenu statique	5
Qualité et sécurité	Organisation, htmlspecialchars, absence de debug	2
Questions de compréhension	Réponses précises et vocabulaire technique	2

7. Livrables

- Le dossier complet du projet food compressé au format ZIP.
- Le fichier base.sql permettant de recréer la base et les données.
- Une capture de la page montrant les catégories chargées depuis MySQL.
- Un court fichier README indiquant le nom de la base et la procédure de lancement.

BONUS FACULTATIF - AJAX : charger ou filtrer les catégories sans recharger toute la page. Ce bonus ne remplace pas l'implémentation PHP/PDO obligatoire.

Important : une donnée modifiée dans MySQL doit modifier l'affichage sans changement du code HTML/PHP.